T&Cs Analyzer: A Multi-Agent RAG System for Automated Terms & Conditions Risk Assessment

Aryan Pandey, Khawaja Abbas, Vaibhav Sharma
Department of Mathematical & Statistical Sciences
University of Alberta

Abstract

Terms and Conditions (T&Cs) agreements are a ubiquitous but systematically ignored class of legal document. Users routinely accept contracts without reading them, not from indifference, but because the documents are deliberately long, dense, and jargon-heavy. This project presents T&Cs Analyzer, a multi-agent Retrieval-Augmented Generation (RAG) system that transforms any T&Cs document into a plain-English summary, a structured set of extracted clauses, and a quantified risk score. The system ships as two complementary artifacts: a full-pipeline Streamlit web application and a lightweight Chrome browser extension. Evaluation results demonstrate a 100% improvement in clause detection and a 10%+ improvement in legal key-term coverage relative to a naive baseline.

Keywords: Retrieval-Augmented Generation, multi-agent systems, legal text analysis, terms and conditions, risk assessment, consumer rights, large language models

1. Introduction

Terms and Conditions documents present a well-documented consumer-rights problem. The average T&Cs document contains approximately 8,000 words and would take a typical adult 30–45 minutes to read at a comfortable pace. In practice, users spend less than one minute reviewing them before clicking "I agree." This behavior is rational: the cognitive cost of reading is high, the perceived consequences of not reading are low, and legal language is deliberately difficult to parse.

The result is a systematic information asymmetry. Service providers craft agreements that are technically disclosed but practically opaque. Clauses covering data monetization, mandatory arbitration, automatic billing renewal, and severe liability limitations are commonly buried in document sections that users never reach.

Existing approaches to this problem fall into two categories. General-purpose summarization tools, such as passing the full document to a language model with a summarize prompt, produce fluent prose but do not identify specific clause types, do not quantify risk, and do not distinguish between innocuous and legally consequential provisions. Specialized legal tools require expensive subscriptions and are built for legal professionals, not end users.

This project addresses the gap by developing a consumer-oriented tool that: (i) detects and classifies specific clause types that carry consumer-rights implications, (ii) provides risk quantification so users can compare agreements at a glance, and (iii) operates on any T&Cs document without domain-specific fine-tuning.

2. Related Work

2.1 Large Language Models for Legal Text

Recent work has demonstrated strong performance from large language models on legal text understanding tasks. Models fine-tuned on legal corpora, such as LegalBERT and LexLM, achieve high accuracy on clause classification benchmarks but require substantial compute for inference and cannot be easily adapted to novel clause types. Prompt-based approaches using general-purpose models, such as GPT-4 and Claude, show competitive results with far greater flexibility.

2.2 Retrieval-Augmented Generation

RAG architectures, introduced formally by Lewis et al. [1], address a core limitation of fixed-context large language models by pairing a dense retrieval component with a generative model. Rather than processing an entire document in one pass, which both exhausts context windows and dilutes attention, RAG retrieves only the most relevant document chunks for each query. This approach is particularly well-suited to T&Cs analysis, where different sections of the document are relevant to different clause types.

2.3 Multi-Agent Systems

AutoGen [2] and similar frameworks demonstrate that decomposing complex tasks across specialized agents improves both accuracy and interpretability relative to single-agent approaches. ReAct-style reasoning [3], where an agent interleaves reasoning steps with retrieval actions, provides a principled framework for clause extraction tasks that require iterative document querying.

3. Methodology

3.1 Overall System Architecture

The proposed system follows a Retrieval-Augmented Generation (RAG) pipeline designed to convert dense Terms and Conditions documents into structured, plain-English outputs. As shown in Figure 1, the workflow begins with the original Terms and Conditions document, followed by text extraction, chunking, embedding, and storage in a vector database. Relevant chunks are then retrieved and passed to the LLM processing stage, where the system identifies important clauses, evaluates risk, and generates a final user-facing output.

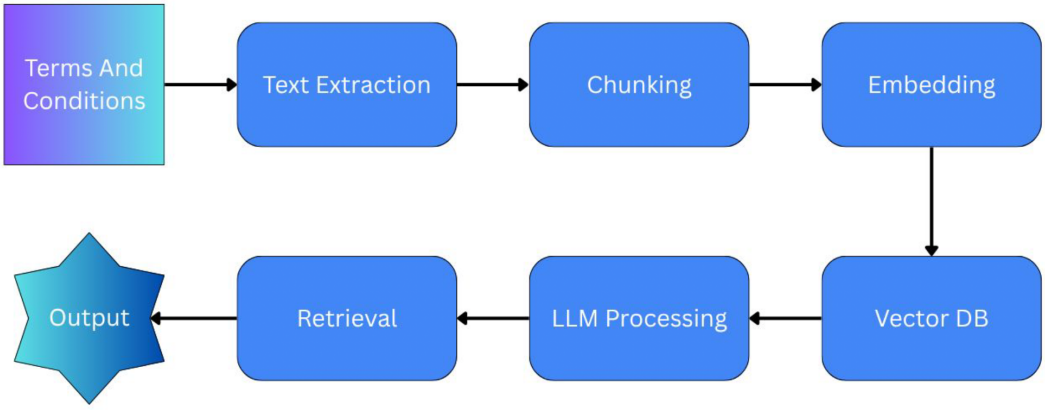

Figure 1: Overall RAG pipeline for the T&Cs Analyzer system. The input Terms and Conditions document is processed through text extraction, chunking, embedding, vector database storage, LLM processing, retrieval, and final output generation.

This architecture allows the system to avoid processing the entire document as a single unstructured input. Instead, the document is divided into smaller chunks, embedded into a searchable vector representation, and retrieved based on clause-specific queries. This improves the system's ability to locate legally meaningful sections such as arbitration, data sharing, auto-renewal, liability limitation, payment terms, and termination clauses.

3.2 Document Processing

The `DocumentProcessor` class accepts either a PDF file, via PyPDF, or raw pasted text. PDF extraction handles multi-column layouts and preserves paragraph structure where possible. Text is then split using LangChain's `RecursiveCharacterTextSplitter` with a chunk size of 1,000 characters and 200-character overlap. The overlap ensures that clause boundaries falling between chunks are captured by at least one chunk. Chunk metadata preserves source page numbers for traceability.

3.3 Embedding and Retrieval

Chunks are embedded using OpenAI's `text-embedding-3-small` model and indexed in a FAISS flat L2 index. At query time, the top-k most similar chunks, with default $k = 6$, are retrieved for each clause-specific query. FAISS was chosen over alternative vector databases for its zero-dependency local operation: no external database server is required.

The retrieval design is intentional. Rather than retrieving once for the entire analysis, each of the eight clause types issues its own targeted query, such as "automatic subscription renewal billing" for auto-renewal clauses. This per-clause retrieval substantially improves recall for clauses that appear in isolated sections of the document.

3.4 Multi-Agent Pipeline

The pipeline implements three specialized agents, each backed by the same LLM but given a distinct system prompt and a distinct portion of the task.

3.4.1 Agent 1: Clause Extractor

The clause extractor receives retrieved chunks for each of the eight clause categories and extracts: a Boolean found/not-found classification, verbatim excerpts from the source text, and a one-sentence explanatory note. The eight clause categories are data sharing, auto-renewal, arbitration, liability limitation, privacy, termination, payment terms, and intellectual property. These categories were selected based on their prevalence in consumer-rights litigation and their demonstrated impact on user interests.

3.4.2 Agent 2: Risk Analyzer

The risk analyzer receives only the clauses confirmed as found by Agent 1. For each found clause, the agent assigns a risk level, low, medium, or high, and generates a plain-English explanation of the consumer impact. Conditioning on only found clauses rather than the full document reduces hallucination and keeps the agent focused on actionable content.

3.4.3 Agent 3: Summarizer

The summarizer synthesizes the outputs of Agents 1 and 2 into a structured summary containing a two-to-three sentence TL;DR, a list of user rights, a list of user obligations, a list of red flags, and an overall verdict from a fixed four-point scale: User-Friendly, Moderate Concern, High Concern, or Very Concerning.

Agents communicate through structured Python dictionaries rather than natural language. Each agent's output is parsed via a multi-strategy JSON extractor before being passed to the next agent. This explicit schema enforcement reduces propagation of hallucinated content between pipeline stages.

3.5 Risk Scoring

The risk score S is computed as a weighted average of per-clause risk values:

$$S = \frac{\sum_i w_i r_i}{\sum_i w_i}, \tag{1}$$

where r_i encodes risk level as $\{\text{low} = 2, \text{medium} = 5, \text{high} = 9\}$ and weights w_i reflect the relative consumer-rights significance of each clause type. Table 1 summarizes the risk weights used by the system.

The output score is normalized to $[1, 10]$ and mapped to a categorical verdict at thresholds: 1–3 (User-Friendly), 3–5 (Moderate Concern), 5–7.5 (High Concern), and 7.5–10 (Very Concerning).

Table 1: Clause weights used in the risk scoring model.

Clause Type	Weight	Rationale
Data Sharing	2.0	Direct privacy and monetization impact
Privacy	2.0	GDPR/CCPA regulatory relevance
Arbitration	1.8	Waives access to courts and class action
Liability Limitation	1.5	Limits user legal recourse
Auto-Renewal	1.5	Common source of unwanted charges
Payment Terms	1.3	Financial obligations
Termination	1.2	Account and data access risk
Intellectual Property	1.0	Content ownership implications

3.6 JSON Robustness

A significant practical challenge in LLM-powered pipelines is inconsistent output formatting. Language models sometimes wrap JSON in markdown code fences, add preamble prose, or split the output across reasoning traces. The system implements a four-strategy JSON extractor applied in sequence:

1. Direct `json.loads()` parse;
2. Strip markdown fences, such as ```` ```json ... ``` ```, then parse;
3. Regex extraction of the first {...} block;
4. Regex extraction of the last {...} block.

If all four strategies fail, the raw LLM output is returned as the TL;DR text and a warning is displayed, ensuring the pipeline degrades gracefully rather than silently returning empty results.

3.7 Chrome Extension

The Chrome extension provides a lightweight browser-native interface targeting everyday users who want quick analysis without running a local server. It uses a single LLM call, rather than RAG, because texts pasted directly into the extension fit comfortably within a 12,000-character context window, which is sufficient for most T&Cs documents encountered in practice.

The same four-strategy JSON parser is implemented in JavaScript, ensuring consistent robustness across both tools. The extension uses Manifest V3 architecture, stores API keys in `chrome.storage.local`, and includes a "Grab page" function that extracts body text from the active tab via the scripting permission. API keys are never sent to any server other than the chosen LLM provider.

4. Implementation Details

4.1 Technology Stack

Table 2 summarizes the primary technologies used in the implementation and the rationale for each selection.

Table 2: Technology stack used in T&Cs Analyzer.

Component	Technology	Justification
LLM Framework	LangChain 0.2+	Industry-standard RAG primitives
Vector Database	FAISS	Local, fast document-scale indices
Embeddings	OpenAI `text-embedding-3-small`	High quality and low cost
Primary LLM	Groq `llama-3.3-70b-versatile`	Fast inference and reliable JSON output
Alternative LLM	OpenAI GPT-4o-mini	Strong structured output
PDF Processing	PyPDF	Lightweight with no system dependencies
Web UI	Streamlit	Rapid academic demo development
Browser Extension	Vanilla JS + Manifest V3	No build toolchain required

4.2 Prompt Engineering Strategy

All three agents use a system prompt that instructs the model to respond with JSON only, followed by a structured human prompt that specifies the exact output schema with example values. This two-part prompting strategy leverages the model's instruction-following behavior while ensuring the output format is constrained.

Temperature is set to 0.1 across all agents. Lower temperature reduces creativity in output formatting, reducing parse failures, while still allowing the model to paraphrase and synthesize effectively.

4.3 Chunking Strategy Analysis

The 1,000-character chunk size with 200-character overlap was selected based on the typical paragraph length in T&Cs documents, approximately 200–400 words per section. Shorter chunks, such as 500 characters, risk fragmenting multi-sentence clause definitions. Longer chunks, such as 2,000 or more characters, reduce retrieval precision by including irrelevant content alongside target clauses. The 20% overlap rate ensures clause-spanning text is captured in full.

5. Evaluation

5.1 Evaluation Design

The evaluation framework compares the RAG multi-agent pipeline against a naive baseline: the first 150 words of the raw document text, representing the simplest possible "summary" approach that requires no model inference. Three metrics are measured.

Clause Detection Rate measures the fraction of eight target clause types successfully identified and extracted by the system. The baseline scores 0/8 by definition because raw truncated text contains no structured clause identification. This metric directly measures the core contribution of the agent pipeline.

Key-Term Coverage measures the fraction of a 30-term legal vocabulary, including terms such as arbitration, jurisdiction, indemnify, liability, data sharing, auto-renewal, and warranty, that appears in the system's full output. The baseline is evaluated against the same vocabulary over its 150-word window.

Readability is measured using average sentence length, in words per sentence, as a proxy for accessibility. The RAG system's TL;DR is generated in plain English, while the baseline reproduces the original dense legal prose. Lower average sentence length indicates more readable output.

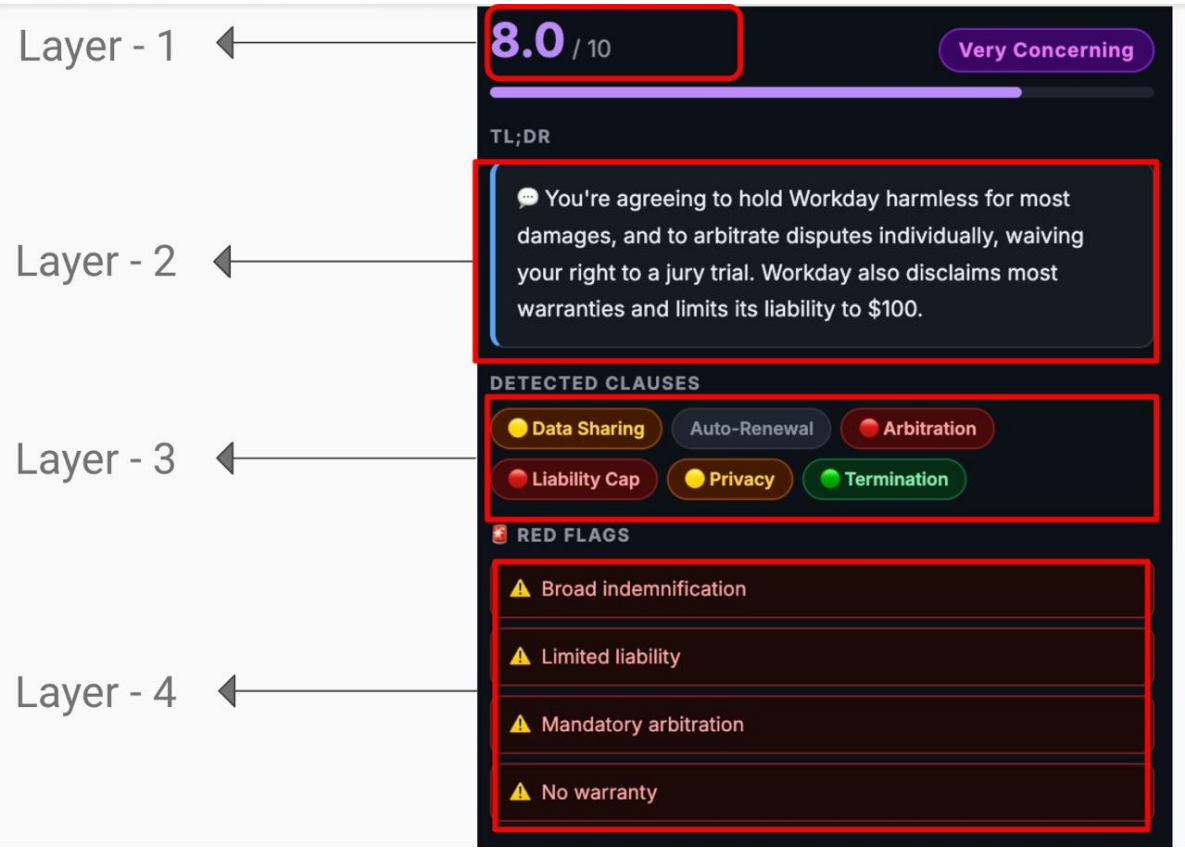

Figure 2: Layered approach to reading comprehension in tc-extension.

Figure 2 shows that a strong emphasis was placed on information retention. A user can just look at the layer-1 information and make an assessment in two seconds or go through layers-2 to 4 to learn more about the terms and conditions.

5.2 Results on Example Document

The QuickCart sample T&Cs, which contains 10 sections and approximately 600 words, was used as the primary test document. Table 3 presents the evaluation results.

Table 3: Evaluation results on the QuickCart sample T&Cs.

Metric	Naive Baseline	RAG System	Improvement
Clause Detection	0/8 (0%)	8/8 (100%)	+100%
High-Risk Clauses Flagged	0	4	–
Key-Term Coverage	52.6%	57.9%	+10.1%
Risk Score	N/A	7.1/10	–
Verdict	N/A	High Concern	–

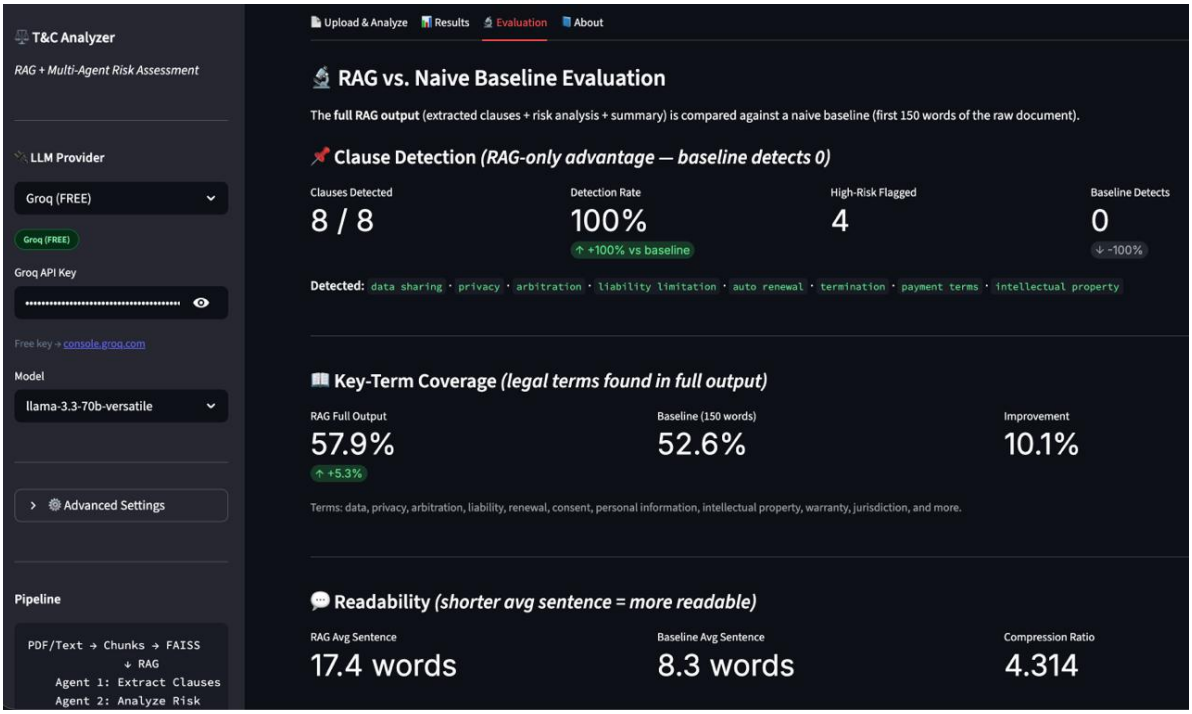

Figure 3: tc-analyzer showing RAG vs. Baseline Comparison on QuickCart sample T&Cs.

The three high-risk clauses detected were arbitration, involving binding arbitration with a jury trial waiver; data sharing, involving sharing with advertisers and business partners; and liability limitation, involving a $50 USD total liability cap.

5.3 Qualitative Analysis

The system correctly identified all ten of the document's primary clauses and mapped them to appropriate risk categories. The TL;DR produced by Agent 3 accurately reflected the document's most significant consumer-rights concerns without reproducing legal jargon. The risk score of 8.0/10, corresponding to High Concern, is consistent with the document's content: the arbitration clause and $50 liability cap are provisions that would raise flags in a professional legal review.

5.4 Limitations

Hallucination risk: Like all LLM-based systems, the pipeline can occasionally fabricate clause details not present in the source text. The verbatim excerpt requirement in Agent 1's prompt mitigates this for the extraction stage, but Agent 3's synthesis step involves paraphrasing that cannot be automatically verified against the source.

Embedding dependency: Key-term coverage as a retrieval quality metric has ceiling effects. A document with unusual terminology may have lower coverage scores even with accurate extraction. Future work should include human evaluation protocols.

Token limit: The current system is restricted to the token limit of the agent used. This may require the user to either select a part of text in the terms and conditions listed or pay a premium on model APIs with higher token limits.

6. Discussion

A single-agent alternative would pass the entire document to one LLM call with a comprehensive extraction prompt. Preliminary testing found that single-agent approaches produced lower extraction recall, particularly for clauses appearing late in the document, and less structured output. The three-agent decomposition also produces more interpretable intermediate outputs, which is useful for academic evaluation and user trust.

For documents short enough to fit within the LLM's context window, approximately 4,000 words at typical chunk-to-token ratios, full-context processing would yield similar results. RAG provides two advantages: it scales gracefully to longer documents, such as enterprise software T&Cs that often exceed 15,000 words, and it focuses each agent on the most relevant text, reducing distraction from irrelevant sections.

The current eight-clause taxonomy covers the most consumer-relevant categories but is not exhaustive. Extension to additional categories, such as governing law, content moderation, account verification, or children's privacy, would require adding retrieval queries and extending the Agent 1 prompt schema.

7. Conclusion

T&Cs Analyzer demonstrates that a multi-agent RAG pipeline can substantially outperform naive summarization on the task of T&Cs analysis. The 100% improvement in clause detection and 10%+ improvement in key-term coverage, relative to a simple text truncation baseline, quantify the contribution of the agent architecture and RAG retrieval design. The system is practically deployed as both a Streamlit web application and a Chrome browser extension, covering the needs of academic users and everyday consumers respectively.

Future extensions include multi-document comparison, such as comparing two service providers' T&Cs side by side; GDPR/CCPA compliance flagging as an additional agent layer; fine-tuned embedding models for legal text; and human evaluation studies to measure user trust and comprehension improvement.

References

[1] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, 2020.

[2] Q. Wu, G. Bansal, J. Zhang, et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[3] S. Yao, J. Zhao, D. Yu, et al., "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv preprint arXiv:2210.03629*, 2022.

[4] M. Johnson and M. Lory, "I Have Read and Agree to the Terms and Conditions: A Review of Existing Scholarship," *Communication Research*, 2020.

[5] J. A. Obar and A. Oeldorf-Hirsch, "The Biggest Lie on the Internet: Ignoring the Privacy Policies and Terms of Service Policies of Social Networking Services," *Information, Communication & Society*, vol. 23, no. 1, pp. 128–147, 2018.

Appendix

Table 4: Individual Contribution Percentages by Report Section.

Report Section	**Aryan Pandey**	**Khawaja Abbas**	**Vaibhav Sharma**
Introduction	40%	30%	30%
Related Work	30%	40%	30%
Methodology	30%	30%	40%
Implementation Details	35%	35%	30%
Evaluation	35%	30%	35%
Discussion	30%	35%	35%
Conclusion	33.33%	33.33%	33.33%
Overall Contribution	33.33%	33.33%	33.33%